

Explanation of Commented Parts in the Node.js Dictionary Server

Why `https` is required

```
const https = require("https"); // application is acting as a client to
the api side because dictionary_api is https
```

This application acts as an **HTTP server** for the browser, but it also acts as an **HTTPS client** when it calls the Dictionary API.

Because `dictionaryapi.dev` uses HTTPS (TLS-encrypted communication), Node.js requires the `https` module. The `http` module cannot make secure API requests.

Environment Variables (`process.env.PORT`)

```
const port = process.env.PORT || 3000; // if a local port is not provided
we use 3000
// what are environmental variables? key value pairs that are set outside
of the application usually in the operating system
```

Environment variables are **key–value pairs defined outside the application**, typically by:

- the operating system
- a terminal session
- a cloud hosting platform (Render, Heroku, etc.)

They allow configuration without changing source code. If a hosting provider supplies a `PORT`, the server uses it. Otherwise, the server defaults to `3000` for local development.

`req` and `res` Objects

```
function request_handler(req, res) { // creates req and res
```

For every incoming HTTP request:

- `req` (request) contains information sent **from the client**, such as the URL, headers, and method.
- `res` (response) is the object used to send data **back to the client**.

These objects are created by Node's HTTP server and passed into the handler automatically.

Why the callback in `https.request()` is optional

```
const dictionary_api = https.request(dictionary_url); // why is the
callback in .request optional
```

The callback is optional because `https.request()` returns a **ClientRequest object**, which is also an **EventEmitter**.

Instead of passing a callback directly, this code listens for events explicitly:

```
dictionary_api.once("response", ...)
dictionary_api.once("error", ...)
```

This event-driven approach provides more control and matches Node's asynchronous design model.

Aborting a request on timeout

```
dictionary_api.setTimeout(5000, function () {
  dictionary_api.destroy(); // aborts the request
});
```

If the API does not respond within 5 seconds:

- the request is forcefully terminated using `destroy()`
- the server responds with a **504 Gateway Timeout**

This prevents the server from hanging indefinitely while waiting on an external service.

Why `res` is passed through multiple functions

```
res.end(results_html); // response back to the client/user
```

The `res` object represents the **original client response**.

Even though the Dictionary API call is asynchronous and handled in other functions, the same `res` object must be preserved so the server can eventually send a response back to the user.

This pattern is common in Node.js for coordinating asynchronous workflows that end in a single HTTP response.